Dominó Caos: aplicación Android multijugador con arquitectura MVVM y sincronización en tiempo real con Firebase

Cristian Castro Tapiero
Facultad Tecnológica
Universidad Distrital Francisco José de Caldas
Bogotá, Colombia
Código: 20222578114
ccastrot@udistrital.edu.co

Daniel Esteban Pardo Moreno
Facultad Tecnológica
Universidad Distrital Francisco José de Caldas
Bogotá, Colombia
Código: 20222578072
depardom@udistrital.edu.co

Fidel Esteban Beltrán Cobos
Facultad Tecnológica
Universidad Distrital Francisco José de Caldas
Bogotá, Colombia
Código: 20222578061
febeltranc@udistrital.edu.co

***Resumen*—Este artículo presenta el diseño e implementación de Dominó Caos, una aplicación Android desarrollada en Kotlin con Jetpack Compose para jugar dominó multijugador (2 a 4 jugadores) en salas sincronizadas en tiempo real. A las reglas clásicas del dominó doble seis se agregan cuatro poderes de uso limitado y cinco eventos aleatorios que alteran turnos, fichas, puntaje o pozo. La aplicación sigue el patrón Model-View-ViewModel (MVVM): un motor de juego en Kotlin puro e inmutable, repositorios que se comunican con Firebase Authentication y Firebase Realtime Database, ViewModels que exponen el estado mediante StateFlow y una interfaz construida con componentes reutilizables. Cada jugada se aplica dentro de una transacción de Firebase, lo que garantiza un estado único y consistente entre dispositivos. La lógica se validó con 35 pruebas unitarias y 5000 partidas simuladas.**

***Palabras clave*—Android, Kotlin, Jetpack Compose, MVVM, Firebase, Realtime Database, programación por componentes, videojuegos multijugador.**

I. Introducción

El Taller 02 de Programación por Componentes plantea construir una aplicación Android en Kotlin para jugar dominó "Caos": una variante en la que el estado de la partida cambia mediante poderes y eventos. Los requisitos incluyen inicio de sesión, sincronización en tiempo real con Firebase, validación de movimientos, cálculo de puntaje, detección automática del fin de la partida, componentes reutilizables y una arquitectura MVVM que separe la lógica de la interfaz.

Este documento describe las especificaciones del diseño (Sección II), la arquitectura MVVM (Sección III), el diagrama de clases (Sección IV), la arquitectura de componentes e integración con Firebase (Sección V), las pruebas (Sección VI) y las conclusiones (Sección VII).

II. Especificaciones del diseño

A. Requisitos funcionales

La Tabla I relaciona cada requisito del taller con el elemento del sistema que lo implementa.

TABLA I
Requisitos y su implementación

Requisito	Implementación
Login y creación de cuenta	`AuthRepository`, `AuthViewModel`, pantallas de login y registro
Sala compartida	Código de 4 caracteres, crear/unirse, sala de espera
Tiempo real	`GameRepository.observeRoom()` con `ValueEventListener`
7 fichas y turnos	`DominoRules.deal()`, `TurnManager.next()`
Validación de movimientos	`DominoRules.canPlace()`, `GameEngine.playTile()`
Robar / perder turno	`GameEngine.drawTile()`, `passTurn()`
Poderes con usos limitados	`PowerType`, `PowerManager`
Eventos aleatorios	`ChaosEvent`, `EventGenerator`
Puntaje	`ScoreCalculator`
Fin automático y desempate	`GameEngine.finish()`, `determineWinner()`
Abandonar y reiniciar	`GameEngine.abandon()`, `restart()`
Turno y puntaje visibles	Componente `PlayerBadge`

B. Reglas del juego

Se usa el dominó doble seis (28 fichas). Cada jugador recibe 7 fichas y el resto forma el pozo. Comienza quien tenga el doble más alto. En su turno, el jugador selecciona una ficha y la coloca en el extremo izquierdo o derecho del tablero si uno de sus valores coincide con ese extremo. Si no tiene movimiento válido debe robar del pozo; si el pozo está vacío, pierde el turno. Una ronda equivale a un turno por jugador y la partida dura 5, 10 o 15 rondas.

C. Poderes

Cada jugador dispone de un uso de cada poder (Tabla II). El poder se selecciona antes de mover y solo se permite uno por turno.

TABLA II
PODERES ESPECIALES

Poder	Efecto
Comodín	La siguiente ficha se coloca sin coincidir con el extremo
Salto	El siguiente jugador pierde su turno
Intercambio	Cambia la ficha seleccionada por una aleatoria del oponente
Inversión	Invierte el sentido de los turnos

D. Eventos aleatorios

Al cerrar cada turno existe una probabilidad del 25 % de que ocurra uno de los eventos de la Tabla III.

TABLA III
EVENTOS ALEATORIOS

Evento	Efecto	Afecta
Tormenta de fichas	Todos roban una ficha	Fichas, pozo
Doble puntaje	La siguiente jugada vale el doble	Puntaje
Giro caótico	Se invierte el sentido	Turno
Recarga de poder	Recupera un uso de un poder	Poderes
Impuesto al líder	El líder pierde 5 puntos	Puntaje

E. Puntaje y fin de la partida

Los puntos de una jugada son la suma de los valores de la ficha más un bono igual a la suma de los extremos del tablero cuando esta es múltiplo de 5; con el evento Doble puntaje el total se duplica. Quien se queda sin fichas suma además los puntos que quedan en las manos rivales.

La partida termina automáticamente cuando un jugador se queda sin fichas, cuando ningún jugador puede mover y el pozo está vacío, o al superar el número máximo de rondas. Gana el mayor puntaje; en caso de empate gana quien haya usado menos poderes.

F. Requisitos no funcionales

- Android 7.0 (API 24) o superior, orientación vertical.
- Kotlin 2.2.10, Jetpack Compose (BOM 2026.02.01), Android Gradle Plugin 9.3.2.
- Firebase BoM 34.0.0: Authentication y Realtime Database (plan Spark).
- Solo usuarios autenticados acceden a las salas; cada usuario solo accede a su historial.

III. ARQUITECTURA MVVM

La aplicación se organiza en el patrón Model-View-ViewModel. Cada capa solo conoce a la capa inferior y solo la capa de datos se comunica con Firebase (Fig. 1).

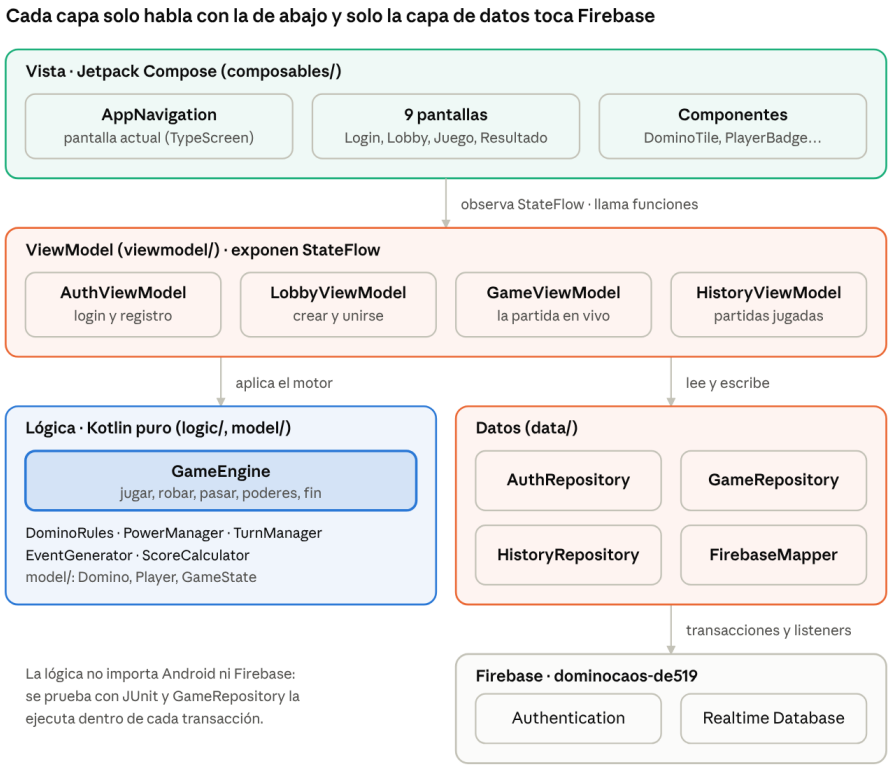


Fig. 1. Arquitectura MVVM de Dominó Caos. La vista observa el estado de los ViewModels; estos aplican el motor de juego (lógica) y leen o escriben mediante los repositorios (datos), únicos que acceden a Firebase.

A. Vista (View)

Las funciones `@Composable` de `composables/` no contienen lógica de juego. Cada pantalla recibe un objeto `UiState` inmutable y expone callbacks para las acciones del usuario. La navegación usa un estado `currentScreen` del tipo `enum TypeScreen`, siguiendo la metodología del proyecto de clase HockeyGame.

B. ViewModel

Los cuatro ViewModels (`AuthViewModel`, `LobbyViewModel`, `GameViewModel`, `HistoryViewModel`) guardan el estado de su pantalla en un `MutableStateFlow` y lo exponen como `StateFlow`. La vista lo observa con `collectAsStateWithLifecycle()`. Cada acción del usuario se traduce en una llamada a un repositorio que ejecuta la función correspondiente del motor.

C. Modelo

El modelo tiene dos partes. La lógica (`model/` y `logic/`) está escrita en Kotlin puro, sin dependencias de Android ni de Firebase, con funciones puras que reciben un `GameState` y devuelven uno nuevo. La capa de datos (`data/`) persiste y sincroniza ese estado en Firebase mediante repositorios y un mapeador.

Esta separación permite probar la lógica con JUnit sin emulador y reutilizar las mismas funciones del motor dentro de las transacciones de Firebase.

IV. Diagrama de clases

La Fig. 2 muestra las doce clases del núcleo. `GameState`, `Player` y `Domino` son `data class` inmutables: ninguna jugada modifica el estado existente, sino que `GameEngine` devuelve una copia nueva encapsulada en un `MoveResult` sellado (`Ok` o `Invalid` con su `MoveError`).

`GameEngine` coordina cinco clases de reglas: `DominoRules` (mazo, reparto y validación), `PowerManager` (poderes y sus usos), `EventGenerator` (eventos aleatorios), `ScoreCalculator` (puntaje y ganador) y `TurnManager` (siguiente turno según el sentido). Las clases marcadas como «`object`» son singletons sin estado; `GameEngine` y `EventGenerator` reciben un `Random` en el constructor para que las pruebas usen una semilla fija.

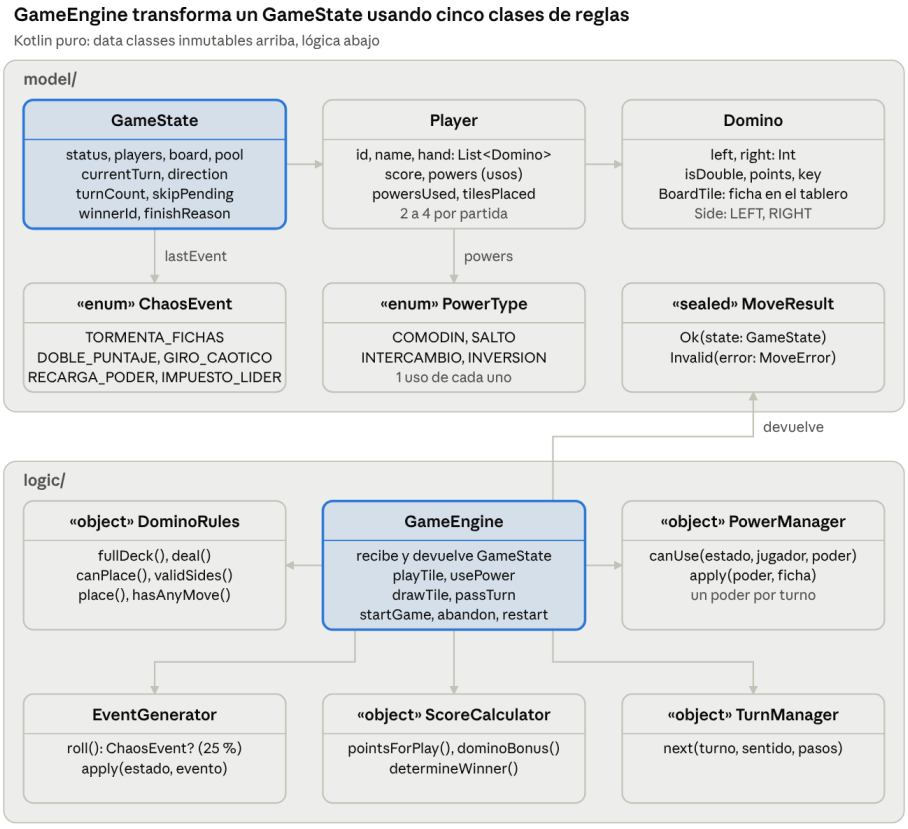


Fig. 2. Diagrama de clases de los paquetes model/ y logic/. Las flechas indican uso o composición; GameEngine recibe y devuelve GameState a través de MoveResult.

Las clases de datos y presentación completan el modelo: `FirebaseMapper` convierte `GameState` a mapas compatibles con Firebase; `AuthRepository`, `GameRepository` y `HistoryRepository` encapsulan el acceso a Firebase; y los ViewModels de la Sección III los consumen.

V. Arquitectura y componentes

A. Componentes reutilizables

La interfaz se compone de componentes que reciben datos por parámetro y notifican eventos mediante lambdas (Tabla

IV). `UiMappings` traduce los enumerados de la lógica a textos, íconos y colores, de modo que la lógica no depende de recursos de Android.

TABLA IV
COMPONENTES REUTILIZABLES

Componente	Responsabilidad	Uso
`DominoTile`	Dibuja una ficha con Canvas (orientación, selección, comodín)	Logo, tablero, mano
`BoardView`	Mesa con la cadena de fichas	Juego
`PlayerBadge`	Nombre, puntaje, fichas y turno	Juego
`PowerCard`, `PowerSelectionDialog`	Selección de poderes y usos	Juego
`HandView`, `GameControls`	Mano y botones de jugada	Juego
`EventDialog`, `StatusChip`	Eventos y efectos activos	Juego
`PrimaryButton`, `AppTextField`, `ScreenHeader`, `MenuCard`	Controles comunes	Todas

B. Pantallas

La aplicación tiene nueve pantallas: inicio de sesión, registro, menú principal, crear partida, unirse a partida, sala de espera, juego, resultado e historial. `AppNavigation` cambia de pantalla según las acciones del usuario y según el estado recibido de Firebase: cuando la sala pasa a `PLAYING` todos los jugadores entran al juego, y cuando pasa a `FINISHED` todos ven el resultado.

C. Autenticación con Firebase

`AuthRepository` usa el proveedor de correo y contraseña de Firebase Authentication. `AuthValidator` valida localmente el correo, la contraseña (mínimo 6 caracteres) y el usuario (mínimo 3). Al registrarse se invocan `createUserWithEmailAndPassword` y `updateProfile`, y el perfil se guarda en `users/{uid}/profile`. El `uid` identifica al jugador en las salas y en su historial.

D. Estructura de datos

La Realtime Database (región us-central1) almacena un árbol JSON con dos ramas. En `rooms/{código}` se guarda el `GameState` completo: estado, jugadores con su mano y poderes, tablero, pozo, turno, sentido, último evento y ganador. Las fichas se guardan como texto `"a|b"` para que el tablero sea legible en la consola. En `users/{uid}` se guardan el perfil y el historial de partidas.

E. Sincronización y transacciones

`GameRepository.observeRoom()` registra un `ValueEventListener` sobre la sala y lo expone como `Flow`; cada cambio se propaga a todos los dispositivos conectados. Toda acción del jugador se aplica con `runTransaction`, en cuatro pasos:

1) Firebase entrega el estado actual de la sala.
2) Se ejecuta la función del `GameEngine` sobre ese estado.
3) Si el resultado es `Ok` se escribe el nuevo estado; si es `Invalid` la transacción se cancela y la interfaz muestra el motivo.
4) Si otro dispositivo escribió simultáneamente, Firebase repite el proceso con el estado más reciente.

Así nunca se pierde ni se sobrescribe una jugada y el motor siempre valida contra el estado real del servidor. La Fig. 3 muestra el recorrido completo de una jugada.



Fig. 3. Flujo de una jugada: la pantalla envía la acción, el ViewModel la delega al repositorio, la transacción ejecuta el motor, Firebase guarda el estado y ambos dispositivos se actualizan.

F. Reglas de seguridad

Las reglas de la base de datos permiten leer y escribir salas solo a usuarios autenticados (`auth != null`) y restringen `users/{uid}` a su propietario (`auth.uid == $uid`). Un índice sobre `status` permite consultar eficientemente las partidas en espera.

VI. Pruebas

La lógica se validó con 35 pruebas unitarias JUnit ubicadas en `app/src/test/` (Tabla V), ejecutadas con `./gradlew test`. Adicionalmente se simularon 5000 partidas aleatorias con 2 a 4 jugadores, poderes y eventos activos: todas terminaron, siempre hubo ganador y nunca se perdió ninguna de las 28 fichas.

TABLA V
PRUEBAS UNITARIAS

Clase de prueba	N.º	Verifica
`DominoRulesTest`	7	Mazo, reparto, validación y orientación
`PowerManagerTest`	7	Cada poder, usos y límite por turno
`GameEngineTest`	14	Puntaje, robar, pasar, fin, desempate, abandonar, reiniciar, eventos
`AuthValidatorTest`	3	Login, registro y códigos de sala
`FirebaseMapperTest`	4	Conversión del estado sin pérdida

VII. Conclusiones

Separar el motor de juego en Kotlin puro permitió probar todas las reglas sin emulador y reutilizar las mismas funciones dentro de las transacciones de Firebase. El patrón MVVM, junto con `StateFlow`, hace que un cambio en la base de datos se refleje automáticamente en la interfaz de todos los dispositivos. Las transacciones de Realtime Database resolvieron la concurrencia entre jugadores, y los componentes reutilizables redujeron la duplicación y dieron una interfaz consistente. Como trabajo futuro se propone el inicio de sesión con Google y la reconexión automática de jugadores.

Referencias

[1] Google, "Firebase Realtime Database," Firebase Documentation. [En línea]. Disponible: https://firebase.google.com/docs/database

[2] Google, "Firebase Authentication," Firebase Documentation. [En línea]. Disponible: https://firebase.google.com/docs/auth

[3] Google, "Firebase Android SDK release notes," 2026. [En línea]. Disponible: https://firebase.google.com/support/release-notes/android

[4] Android Developers, "Guide to app architecture," Google. [En línea]. Disponible: https://developer.android.com/topic/architecture

[5] Android Developers, "Jetpack Compose," Google. [En línea]. Disponible: https://developer.android.com/compose

[6] J. E. Hernández R., "Taller 02 – Dominó Caos," Programación por Componentes 301, Universidad Distrital Francisco José de Caldas, Bogotá, 2026.